

QB: Chapter 1.2 — Sorting Techniques

Unit 1 | 5×2 -mark | 5×5 -mark | 3×10 -mark | **Total: 13 questions**

Lectures: L07–L12 | BT Levels: BT1, BT2, BT3, BT4 | COs: CO1, CO2, CO5

Section A: 2-Mark Questions

Q1. [BT1 | CO1, CO5] Define *stable sorting*. Name two sorting algorithms that are stable and two that are unstable. Why does stability matter when sorting records with multiple fields?

Q2. [BT1 | CO1, CO5] State the best-case, average-case, and worst-case time complexity of merge sort, along with its space complexity. Identify the key advantage of merge sort over quick sort.

Q3. [BT2 | CO1, CO5] Explain why quick sort degrades to $O(n^2)$ worst-case time complexity. Describe the specific input condition that triggers this worst case when the last element is always chosen as pivot. How does random pivot selection mitigate this?

Q4. [BT2 | CO2] Describe the difference between row-major and column-major ordering for a 2D array stored in memory. Write the address formula for element $A[i][j]$ in row-major order for an array $A[m][n]$.

Q5. [BT3 | CO2] A 3D float array $A[2][3][4]$ is stored in row-major order with base address $BA = 1000$ and float size = 4 bytes. Derive the address formula for $A[i][j][k]$ and calculate the address of $A[1][2][3]$. Show all working.

Section B: 5-Mark Questions

Q6. [BT3 | CO2] Array $A[3][4]$ is stored in memory with $BA = 3000$ and integer size = 4 bytes.

(a) Using row-major order, calculate the address of $A[2][3]$. (b) Using column-major order, calculate the address of $A[1][2]$. Show derivation of both formulas and all arithmetic steps.

Q7. [BT4 | CO1, CO5] Perform a complete trace of the optimised bubble sort (with early-termination flag) on array $[8, 3, 6, 1, 5]$. Show each pass in a table (index, comparison, swap?, array state). Identify the pass in which the early-termination condition triggers and state the number of comparisons and swaps made in total.

Q8. [BT4 | CO1, CO5] Perform the Lomuto partition step on array $[10, 7, 8, 9, 1, 5]$ with $low = 0$, $high = 5$, and $pivot = arr[high] = 5$. Use a trace table to show, after each value of j , the values of i , j , $arr[j]$, the comparison result, and the array state. State the final position of the pivot and the two sub-arrays to be recursively sorted.

Q9. [BT4 | CO1, CO5] Show the complete recursive decomposition tree of merge sort applied to array [38, 27, 43, 3, 9, 82, 10]. Trace the final merge step that combines sorted halves [3, 27, 38, 43] and [9, 10, 82]. Show each element comparison in the merge table and the resulting merged array.

Q10. [BT4 | CO1, CO5] Merge sort satisfies the recurrence relation $T(n) = 2T(n/2) + cn$. Apply the Master Theorem (Case 2: $a = b^d$) to derive that $T(n) = \Theta(n \log n)$. Identify the values of a , b , and d . Explain why merge sort is preferred over bubble sort for large datasets by computing operation counts for $n = 1024$ under each algorithm's worst case.

Section C: 10-Mark Questions

Q11. [BT4 | CO1, CO5] (*Experiment E4 — Kth Largest Element, LeetCode #215*)

The problem requires finding the k -th largest element in an unsorted array without fully sorting it.

(a) Describe the QuickSelect algorithm — a variant of quick sort's partition step — that finds the k -th largest (or smallest) element in $O(n)$ average time. Write the partition logic and the selection recursion clearly distinguishing the base case from the recursive cases.

(b) Trace QuickSelect on array [3, 2, 1, 5, 6, 4] to find the 2nd largest element ($k = 2$). At each step, show the pivot chosen (last element, Lomuto scheme), the array after partitioning, and the sub-array selected for the next recursive call.

(c) Compare three approaches to this problem: (i) QuickSelect — $O(n)$ average, $O(n^2)$ worst; (ii) full sort then index — $O(n \log n)$; (iii) min-heap of size k — $O(n \log k)$. Recommend the best approach for each of: $k = 1$, $k = n/2$, and repeated queries on the same array. Justify using complexity analysis.

Q12. [BT4 | CO1, CO5] (*Experiment E2 — Sort an Array, LeetCode #912*)

Given array `nums = [5, 2, 3, 1]`, sort it using two different algorithms.

(a) Implement merge sort. Draw the full recursion tree showing every divide step. Trace each merge operation from the base cases upward, showing the merged result at every level.

(b) Implement quick sort with the Lomuto partition scheme on the same array. Pivot = last element. Show all partition steps, pivot placement, and the recursive sub-calls at each level.

(c) Compare merge sort and quick sort on the following attributes: stability, worst-case time complexity, auxiliary space complexity, suitability for linked lists, and suitability for external sorting (data on disk). For an in-memory sort of 10^6 integers with limited extra memory, recommend one algorithm and justify.

Q13. [BT4 | CO2]

A 3D simulation array `A[4][3][5]` (layers $\times$ rows $\times$ columns) stores double-precision values (8 bytes each) in row-major order at $BA = 2000$.

- (a)** Derive the general row-major address formula for $A[i][j][k]$. Compute the addresses of $A[0][0][0]$, $A[1][2][3]$, and $A[3][2][4]$. Show all intermediate calculations.
- (b)** Define column-major order for a 3D array. Write its address formula for $A[i][j][k]$ (layers $\times$ rows $\times$ columns). Identify one language that natively uses column-major storage and explain why this affects performance in nested loops.
- (c)** Consider `struct S { char a; double b; int c; char d; }` (char = 1 B, int = 4 B, double = 8 B). Compute the size of this struct as declared, accounting for alignment padding. Suggest an optimal field ordering that minimises padding and compute the reduced struct size.